

CSC 1012 Introduction to Computer Programming

Assignment (Individual)

Year 1, Semester 1- 2026



Smart Hospital & Resource Allocation System

Project Report

Name: D. T. Gamage

Index: AS20250644

Submission Date: 20/09/2026

Table of Contents

1. Introduction	3
2. Project Objectives	4
3. Development Timeline	5
4. System Architecture	6
4.1 Module Overview.....	6
4.2 Function List.....	6
5. Data Structure Design	8
6. Main System Components.....	10
6.1 Patient Registration and Input Validation	10
6.4 Age Subsidy	10
6.5 Patient IDs.....	10
7. Core Calculations	11
8. Program Flow	12
9. File Handling	13
10. Test Cases	14
11. Debugging	15
12. Assumptions.....	17
13. Final Output	18
13.1 Menu Option 1	18
13.1 Menu Option 2	19
13.1 Menu Option 3	20
13.1 Menu Option 4	21
13.1 Menu Option 5	22
13.1 Menu Option 6	23
13.1 Menu Option 7	24
.....	24
14. GitHub Repository URL	25
15. Conclusion.....	26

1. Introduction

The Smart Hospital & Resource Allocation System was created as a project for the CSC 1012 Introduction to Computer Programming course and it is a modular system and a menu driven system. The main aim of the project is to showcase the principles of C programming in a hospital setting. The system also has features like patient registration, emergency prioritization, choice of doctor specialty, bed allocation, billing system and assignments of hospital performance.

The project was created from 11 September 2026 to 20 September 2026. Development and testing were done incrementally so that new modules could be tested after completion of each module. This ensures modular programming since multiple source files and header files were used in the development of the project.

The application was developed using Visual Studio Code as the main code editor on a MacBook, and the Apple Clang compiler for C programming. Visual Studio Code was chosen as the development environment as it is a good C programming environment for macOS, whereas Code::Blocks was not available for the macOS system used in this project. The program was compiled and executed using GCC compilation commands in the macOS Terminal. The environment provided the necessary support for developing the whole multi-file C program.

The resulting system is capable of demonstrating variables, data types, operators, branching statements, loops, functions, arrays, strings, pointers, structured arrays, sorting, and mathematical calculations. All of those concepts were applied in the hospital management system project.

The system was developed and tested in modules that were gradually integrated into the main system. Each feature, such as patient validation, bed management, billing calculations, prioritizing patients, producing reports, and preserving records, was tested under different conditions. The method followed made it possible to spot and correct errors when they appeared in the course of the process so that the end product functioned properly as prescribed.

2. Project Objectives

- Utilize one-dimensional and two-dimensional arrays to signify hospitals and patients.
- Implement parallel arrays to keep relevant patients' characteristics.
- Devise functions that can be used repeatedly for actions such as registration, charges, arranging, reporting and file changes.
- Apply loops and conditional statements for the purposes of accomplishing the laws of the hospital.
- Embrace strings when working with either names or identification numbers of patients.
- Show the work of pointers via function inputs and outputs.
- Use selection sort when determining emergency priorities.
- Perform file management for matters relating to bed conditions.

3. Development Timeline

The development of the system has been executed step by step at the time from **September 11 to September 20, 2026**. The development process started with the identification and planning of requirements. Next, the project structure was developed before implementing the core functional software modules, integration, testing, file operations, and final improvements. The GitHub platform was used to create and save development history through meaningful commits. The following table presents the important software development activities done during the project.

Date	Development Stage	Main Activities / Work Completed
11 Sep 2026	Requirement Analysis & Planning	Understood the project requirements, determined the required system features and functionalities, and created the initial project documentation and plan.
12 Sep 2026	Project Initialization & Organization	Set up the project repository and arranged the basic project files and initial project structure.
13 Sep 2026	Documentation & Structuring	Created the README file and Git configuration, and established the source code and documentation folder structure.
14 Sep 2026	System Development & Display	Created declarations for the hospital and display modules, implemented hospital lookup and bed-status display functions, and built and tested the main menu structure.
15 Sep 2026	Patient Data Design	Designed the patient data model using parallel arrays to store and manage patient information.
16 Sep 2026	Patient Registration & Billing	Developed patient registration and bed allocation functionalities, added billing declarations, implemented waiting-time and additional-charge calculations, and connected patient registration with the billing receipt.
17 Sep 2026	Priority Management & Integration	Integrated billing information with the patient registration process, introduced the emergency priority sorting system, added a patient list display option, and integrated priority sorting into the main menu.
18 Sep 2026	Performance Reporting	Completed the reporting module to generate hospital performance reports using patient billing information, total revenue, discounts, and other system data.
19 Sep 2026	File Handling & Data Persistence	Implemented file handling for bed-status persistence and patient record logging, allowing important system data to be stored and retained across program runs.
20 Sep 2026	Final Testing & Refinement	Made final improvements to reinforce correct system functionality, fixed validation and data-handling issues, refined the bill and report output formatting, and finalized the project documentation and report.

4. System Architecture

4.1 Module Overview

The codes are separated into seven modules along with `main.c`, one for each module. Each of these has its own header file containing the functions that can be accessed by other modules. This lets the program's different parts (display, registration, billing, sorting/reporting and file input/output) work independently from one another and comply with the requirements of the task, which state that it is better to break the logic into small pieces instead of adding it directly to the `main()` function.

File	Responsibility
<code>hospital.h</code>	Shared constants used across every module (array sizes, name length).
<code>display.c</code> <code>display.h</code>	Prints the specialty table, ward table, and bed occupancy grid.
<code>patient.c</code> <code>patient.h</code>	Collects patient input, assigns an available bed, and stores the record.
<code>billing.c</code> <code>billing.h</code>	Computes wait time, surcharge, ward cost, discount, and prints the receipt.
<code>sorting.c</code> <code>sorting.h</code>	Sorts registered patients by triage priority.
<code>reports.c</code> <code>reports.h</code>	Aggregates statistics across all registered patients.
<code>fileio.c</code> <code>fileio.h</code>	Saves/loads bed status and appends the patient billing log.
<code>main.c</code>	Owens all data arrays and runs the menu loop that ties the modules together.

4.2 Function List

Function	File	Purpose
<code>displaySpecialties()</code>	<code>display.c</code>	Prints the specialty lookup table.
<code>displayWards()</code>	<code>display.c</code>	Prints the ward lookup table.
<code>displayBedStatus()</code>	<code>display.c</code>	Prints bed occupancy per ward.
<code>registerPatient()</code>	<code>patient.c</code>	Collects input, assigns a bed, triggers billing and logging.

Function	File	Purpose
<code>findAvailableBed()</code>	<code>patient.c</code>	Internal helper: finds the first free bed in a ward (static).
<code>calculateWaitTime()</code>	<code>billing.c</code>	Queue count x avg. consultation time; increments the queue.
<code>calculateSurcharge()</code>	<code>billing.c</code>	Emergency surcharge based on urgency level.
<code>calculateWardCost()</code>	<code>billing.c</code>	Days admitted x ward daily rate.
<code>calculateDiscount()</code>	<code>billing.c</code>	Age-based subsidy discount.
<code>printBillReceipt()</code>	<code>billing.c</code>	Runs the four calculations above and prints the formatted bill.
<code>sortPatientsByPriority()</code>	<code>sorting.c</code>	Selection sort by urgency, tie-broken by registration order.
<code>displaySortedPatients()</code>	<code>sorting.c</code>	Prints the sorted patient list.
<code>countByUrgency()</code>	<code>reports.c</code>	Counts registered patients per urgency level.
<code>calculateTotalRevenue()</code>	<code>reports.c</code>	Sums final bill amounts across all patients.
<code>calculateTotalDiscounts()</code>	<code>reports.c</code>	Sums discount amounts across all patients.
<code>calculateOccupancyPercentage()</code>	<code>reports.c</code>	Occupied beds / capacity, as a percentage, per ward.
<code>findHighestPayingPatient()</code>	<code>reports.c</code>	Finds the index of the largest final bill.
<code>generateSummaryReport()</code>	<code>reports.c</code>	Prints the full analytics report using the functions above.
<code>saveBedStatus()</code> / <code>loadBedStatus()</code>	<code>fileio.c</code>	Persists/restores <code>bedOccupancy[][]</code> to <code>beds_status.txt</code> .
<code>logPatientRecord()</code>	<code>fileio.c</code>	Appends one line per registration to <code>patient_records.txt</code> .

5. Data Structure Design

The Smart Hospital and Resource Allocation System utilize the use of arrays and character arrays to store and manipulate information regarding patients, beds, billing, and hospitals. The parallel arrays make it easy to store related patient information in one index, while the two-dimensional arrays take care of storing occupancy status of beds in various wards. Selection of these data structure solves the requirements of the project while keeping the implementation simple and easy for basic C programming.

Data Structure	Type / Declaration	Purpose	Relevant File(s)
patientIds	char[MAX_PATIENTS][16]	Stores IDs such as PAT-1001.	patient.h, patient.c
patientNames	char[MAX_PATIENTS][NAME_LEN]	Stores patient names.	patient.h, patient.c
patientAges	int[MAX_PATIENTS]	Stores patient ages.	patient.h, patient.c
urgencyLevels	int[MAX_PATIENTS]	Stores urgency levels 1–3.	patient.h, patient.c, sorting.c, reports.c
specialtyIds	int[MAX_PATIENTS]	Stores specialty IDs.	patient.h, patient.c
isAdmitted	int[MAX_PATIENTS]	Stores admission status.	patient.h, patient.c
wardIds	int[MAX_PATIENTS]	Stores assigned ward IDs.	patient.h, patient.c
bedNumbers	int[MAX_PATIENTS]	Stores assigned bed positions.	patient.h, patient.c
daysAdmitted	int[MAX_PATIENTS]	Stores admission duration.	patient.h, patient.c, billing.c
finalBillAmounts	double[MAX_PATIENTS]	Stores final payable bills.	patient.h, patient.c, billing.c, reports.c
discountAmounts	double[MAX_PATIENTS]	Stores discounts applied.	patient.h, patient.c, billing.c, reports.c
bedOccupancy	int[4][20]	Stores bed state: 0	hospital.h, display.c,

		available and 1 occupied.	patient.c, fileio.c
specialtyQueueCount	int[4]	Tracks specialty queues and daily capacity.	patient.c, billing.c
specialtyNames	const char *[4]	Stores fixed doctor specialty names.	main.c
specialtyFees	const double[4]	Stores consultation fees for each specialty.	main.c
consultationTimes	const int[4]	Stores average consultation times.	main.c
specialtyCaps	const int[4]	Stores daily patient limits for specialties.	main.c
wardNames	const char *[4]	Stores fixed ward names.	main.c
wardDailyRates	const double[4]	Stores daily charges for wards.	main.c
wardBedCapacities	const int[4]	Stores the number of beds available in each ward.	main.c, patient.c

6. Main System Components

6.1 Patient Registration and Input Validation

Registration of Patients and Validation of Entries The registration system is responsible for entering the patient's name, their age and urgency level, type of issue, admission status, wards and number of days in the hospital. Integer data requires a combination of functions: `fgets()` and `sscanf()`; statistical analysis is also used. If a name is absent, the information is rejected. The option of assigning the daily limit is verified, and it is only after that changes are made in `patientCount` and `specialtyQueueCount`.

6.2 Bed Allocation

Allocation of the Bed The `bedOccupancy[4][20]` array displays the information about the availability of the bed. Each ward has only the number of beds it has actually, namely: 20, 10, 10 and 5. The bed is assigned to a patient. If the ward is full, the patient is classified as an outpatient.

6.3 Billing

Through the billing module, the computation of the cost of the consultation, along with the financial implication of the emergency surcharge, the cost incurred in the ward, the gross amount due, the age subsidy and the ultimate amount to be paid is carried out. Additionally, the waiting time is worked out through the specialty queue, which is operating.

6.4 Age Subsidy

Every individual less than five or over sixty-five is awarded a subsidy, which amounts to 15% of the total gross bill incurred.

6.5 Patient IDs

PAT-1001 is the format used to assign Patient IDs. This numbering system ensures continuity with every new instance of the program by counting the number of records present in the `patient_records.txt` file.

7. Core Calculations

Calculation	Formula	Relevant File(s)
Waiting Time	Current specialty queue count $\times$ Consultation time	billing.c, patient.c
Emergency Surcharge	Urgent: Base Fee $\times$ 0.20 Critical: Base Fee $\times$ 0.50	billing.c
Ward Cost	Days Admitted $\times$ Ward Daily Rate	billing.c
Gross Bill	Base Fee + Surcharge + Ward Cost	billing.c
Age Subsidy	Gross $\times$ 0.15 if age < 5 or age > 65	billing.c
Final Payable	Gross – Discount	billing.c
Total Revenue	Σ Final Bill Amounts	reports.c
Total Discounts	Σ Discount Amounts	reports.c
Ward Occupancy	$(\text{Occupied Beds} \div \text{Total Beds}) \times 100$	reports.c
Highest-Paying Patient	Maximum value among Final Bill Amounts	reports.c
Bed Availability	bedOccupancy[ward][bed] == 0	patient.c
Bed Allocation	First available bed: $0 \rightarrow 1$	patient.c

8. Program Flow

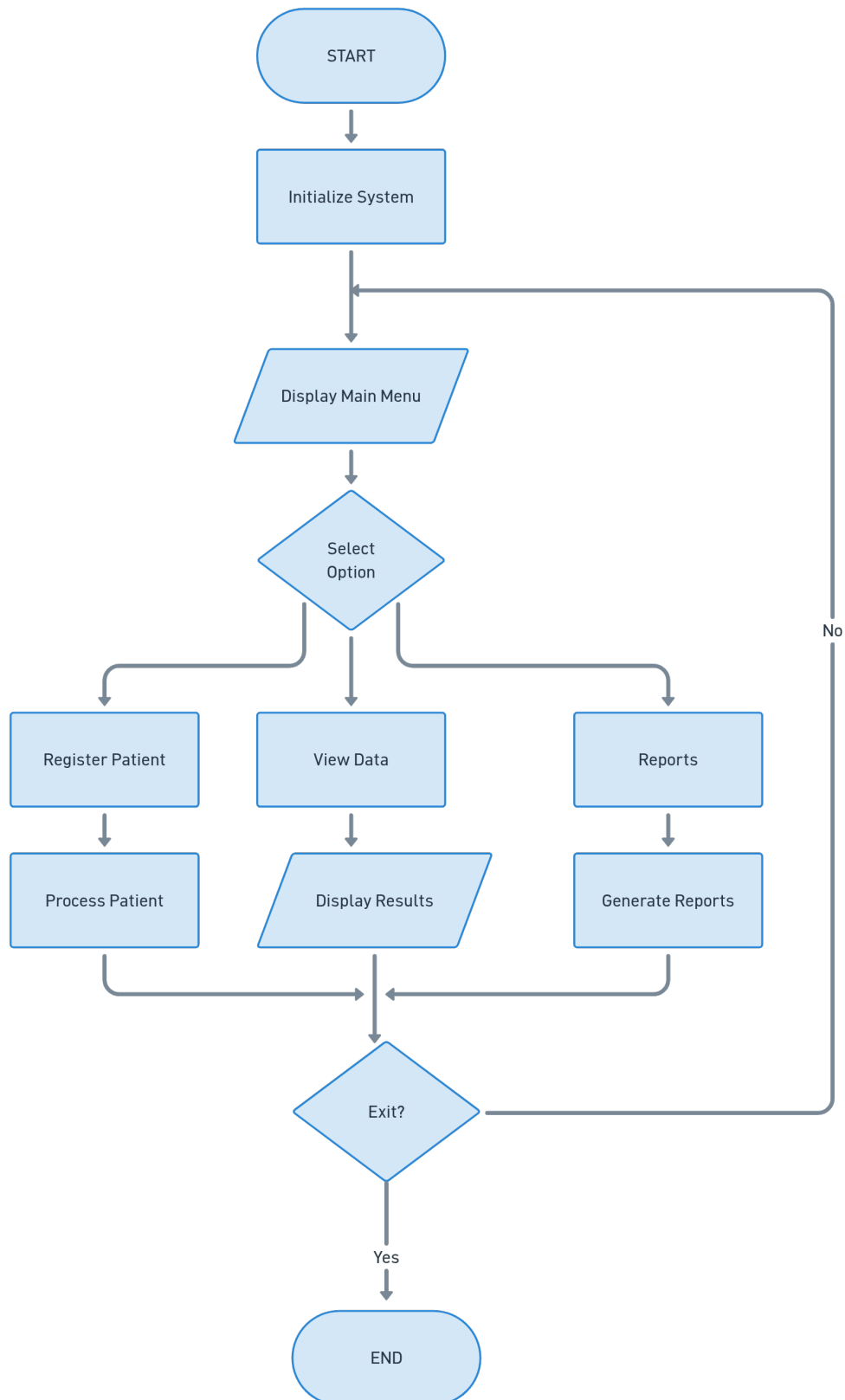


Figure 1: Summary flowchart of the Smart Hospital & Resource Allocation System.

9. File Handling

beds_status.txt stores the current 4×20 bed matrix and is loaded at startup and saved on exit.
patient_records.txt stores permanent patient billing records using append mode.

- beds_status.txt – stores bed occupancy.
- patient_records.txt – appends patient ID, name, urgency level and final payable amount.

Historical records are not reconstructed into current in-memory arrays; they are retained for permanent logging and patient ID continuation.

10. Test Cases

Test Case ID	Test Description	Input / Conditions	Expected Result	Actual Result	Pass/Fail
0001	Register Kamal Perera as a critical patient	Name: Kamal Perera; Age: 70; Urgency: 3 (Critical); Specialty: Cardiology; Admitted: Yes; Ward: ICU; Days: 2	Patient is registered as PAT-1001 and assigned an available ICU bed.	PAT-1001 registered and ICU Bed 01 assigned.	Pass
0002	Calculate waiting time	Cardiology queue count = 0; Consultation time = 30 min	Waiting time = $0 \times 30 = 0$ minutes	0 minutes displayed.	Pass
0003	Calculate emergency surcharge	Base Fee = LKR 4,500; Critical surcharge = 50%	Surcharge = $4,500 \times 0.50 =$ LKR 2,250.00	LKR 2,250.00 displayed.	Pass
0004	Calculate ICU ward cost	ICU rate = LKR 25,000/day; Days = 2	Ward Cost = $2 \times 25,000 =$ LKR 50,000.00	LKR 50,000.00 displayed.	Pass
0005	Calculate gross bill	Base Fee = LKR 4,500; Surcharge = LKR 2,250; Ward Cost = LKR 50,000	Gross = $4,500 + 2,250 + 50,000 =$ LKR 56,750.00	LKR 56,750.00 displayed.	Pass
0006	Calculate age subsidy	Age = 70; Gross = LKR 56,750	Discount = $56,750 \times 0.15 =$ LKR 8,512.50	LKR 8,512.50 displayed.	Pass
0007	Calculate final payable amount	Gross = LKR 56,750; Discount = LKR 8,512.50	Final = $56,750 - 8,512.50 =$ LKR 48,237.50	LKR 48,237.50 displayed.	Pass
0008	Display complete billing receipt	All Kamal Perera details and calculated values	Receipt displays patient, urgency, specialty, ICU bed, charges, discount, final payable and waiting time correctly.	Receipt displayed correctly.	Pass

11. Debugging

Date	Identified Error	Fix or Debugging Done	Relevant Files	Outcome
16 Sep 2026	Invalid or empty patient values could be entered.	Added input validation using <code>fgets()</code> and <code>sscanf()</code> to check the entered values.	<code>patient.c</code>	Invalid values were rejected.
16 Sep 2026	The specialty queue could exceed its daily limit.	Checked the current queue count before registering a new patient.	<code>patient.c</code>	Specialty limits were maintained correctly.
16 Sep 2026	Beds could potentially be assigned beyond the actual ward capacity.	Limited the bed search according to the capacity of the selected ward.	<code>patient.c</code>	Beds were assigned only within the available ward capacity.
17 Sep 2026	Waiting time did not initially account for the number of patients already in the queue.	Calculated the waiting time using the current queue count before increasing the count.	<code>billing.c</code> , <code>patient.c</code>	Waiting time was calculated correctly.
17 Sep 2026	Full wards were not handled properly.	Added a bed-availability check and an outpatient option when no bed was available.	<code>patient.c</code>	Patients could be registered safely even when wards were full.
17 Sep 2026	Patient priority needed to follow urgency level.	Implemented selection sort based on urgency, while keeping registration order for patients with the same urgency.	<code>sorting.c</code>	Patients were displayed in the required priority order.
18 Sep 2026	Report calculations needed to be checked for accuracy.	Tested patient counts, revenue, discounts, ward occupancy, and the highest-paying patient calculations.	<code>reports.c</code>	Reports produced the expected results.
19 Sep 2026	Bed and patient data needed to be preserved	Implemented saving and loading of bed status and patient billing records.	<code>fileio.c</code> , <code>patient.c</code>	Required data was saved successfully.

	between program runs.			
20 Sep 2026	Large monetary values were difficult to read.	Added comma-separated formatting for amounts, such as 4,500.00 and 50,000.00.	billing.c, reports.c	Bill and report amounts became easier to read.
20 Sep 2026	The bill receipt needed clearer separation between sections.	Added horizontal – lines between sections and = lines around the receipt.	billing.c	The receipt became clearer and more organized.
20 Sep 2026	Monetary formatting needed to be consistent in reports and bills.	Reused the same monetary formatting function in the reporting module.	billing.c, reports.c	Bills and reports used a consistent amount format.
20 Sep 2026	Temporary debugging statements caused compilation errors.	Removed temporary printf() statements that had been placed outside functions.	billing.c, reports.c	The program compiled successfully.
20 Sep 2026	The latest code changes were not appearing during testing.	Identified that the older hospital executable was being run instead of the updated smart_hospital executable and recompiled the program.	Build / testing process	The latest source-code changes appeared correctly after recompilation.

12. Assumptions

- The system operates **once per day**, so the specialty queue count and daily specialty limits apply only to the current session.
- Each patient receives a **unique patient ID**, which is assigned automatically in numerical order starting from PAT-1001.
- A patient may choose **only one doctor specialty** during the registration process.
- The number of patients waiting in the queue represents the **number of patients registered in the current session**.
- The waiting time is calculated using the **current queue count × average consultation time** for the selected specialty.
- If the selected ward is full, the patient is **registered as an outpatient** instead of being admitted.
- A bed is assigned based on the **first available bed** in the selected ward.
- The number of beds available is **fixed according to the given hospital requirements**.
- Patients **under 5 years or over 65 years** qualify for a **15% age-based subsidy**.
- An emergency surcharge is applied according to the **urgency level**, namely Normal (0%), Urgent (20%), and Critical (50%).
- Ward charges are calculated by multiplying the **number of days spent in the ward by the daily ward rate**.
- Historical patient records are saved so that **billing records and patient IDs can continue across program runs**, but previous patients are not loaded into the current day's patient records.
- The system assumes that patient information is entered by an **authorized member of hospital staff**.
- The system has **maximum limits for patients, wards, beds, and name length** as defined in the program.

13. Final Output

13.1 Menu Option 1

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 1

ID   Specialty                Fee (LKR)    Time/Patient  Daily Cap
1    General Practice (OPD)    1500.00      15             30
2    Paediatrics               2500.00      20             20
3    Cardiology                4500.00      30             12
4    Neurology                 5000.00      30             10

=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 2: Main menu of the Smart Hospital & Resource Allocation System and option 1 display.

13.1 Menu Option 2

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 2

ID   Ward                                Rate/Day    Capacity
1    General Ward                        3000.00     20
2    Paediatric Ward                    6000.00     10
3    Surgical Ward                     12000.00    10
4    ICU (Intensive Care Unit)         25000.00     5

=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 3 : Menu Option 2 Display

13.1 Menu Option 3

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 3

General Ward: 00000000000000000000 (0/20 occupied)
Paediatric Ward: 0000000000 (0/10 occupied)
Surgical Ward: 0000000000 (0/10 occupied)
ICU (Intensive Care Unit): 00000 (0/5 occupied)

=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 4 : Menu Option 3 Display

13.1 Menu Option 4

```
daha@MacBook-Air Smart-Hospital-System % ./smart_hospital
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 4

Enter patient name: Mr. Kamal Perera
Enter patient age: 70
Enter urgency level (1 = Normal, 2 = Urgent, 3 = Critical): 3
Enter specialty ID (1-4): 3
Is the patient admitted to a ward? (1 = Yes, 0 = No): 4
Please enter a value from 0 to 1.
Is the patient admitted to a ward? (1 = Yes, 0 = No): 1
Enter ward ID (1-4): 4
Enter number of days admitted: 2

=====
                        SMART HOSPITAL ADMISSION & BILL
=====
Patient ID       : PAT-1001
Patient Name    : Mr. Kamal Perera
Age             : 70 Years (15% Subsidy Eligible)
Specialty       : Cardiology
Assigned Ward   : ICU (Intensive Care Unit) (Bed #01)
Urgency Level   : Level 3 (Critical)
=====
Base Consultation Fee : LKR 4,500.00
Emergency Surcharge   : LKR 2,250.00 (50%)
Ward Stay Cost (2 Days) : LKR 50,000.00
=====
Gross Total Bill      : LKR 56,750.00
Age Subsidy Discount  : LKR -8,512.50 (15%)
=====
Final Payable Amount  : LKR 48,237.50
Estimated Waiting Time : 0.00 mins (Immediate Attention)
=====

=====
                        SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 5 : Menu Option 4 Display

13.1 Menu Option 5

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 5

Rank   ID       Name           Urgency
1      PAT-1001  Mr. Kamal Perera  Level 3 (Critical)

=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 6 : Menu Option 5 Display

13.1 Menu Option 6

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 6

=====
HOSPITAL SUMMARY REPORT
=====
Total Patients Registered : 1
- Normal (Level 1)       : 0
- Urgent (Level 2)       : 0
- Critical (Level 3)     : 1
=====
Total Revenue Earned      : LKR 48,237.50
Total Discounts Granted   : LKR 8,512.50
=====
Bed Occupancy by Ward:
- General Ward           : 0.0%
- Paediatric Ward        : 0.0%
- Surgical Ward          : 0.0%
- ICU (Intensive Care Unit) : 20.0%
=====
Highest-Paying Patient    : Mr. Kamal Perera (PAT-1001) - LKR 48,237.50
=====

=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: █
```

Figure 7 : Menu Option 6 Display

13.1 Menu Option 7

```
=====
SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM
=====
1. View Doctor Specialties
2. View Hospital Wards
3. View Bed Occupancy Status
4. Register New Patient
5. View Patients by Priority
6. Generate Summary Report
7. Exit
=====
Enter your choice: 7

Exiting system. Goodbye!
daha@MacBook-Air Smart-Hospital-System %
```

Figure 8 : Menu Option 7 Display

14. GitHub Repository URL

You can find the whole source code, system design, and development documentation of the Smart Hospital & Resource Allocation System in the repository hosted on GitHub. There is everything that is essential for the Smart Hospital & Resource Allocation System including C source and header files, README documentation, and various other project-related files.

Repository URL:

<https://github.com/dahamsat-droid/Smart-Hospital-System.git>

15. Conclusion

The Smart Hospital & Resource Allocation System was developed successfully as a modular, menu-based C program to show the practical application of programming concepts in a hospital setup. It has some basic features including patient registration, emergency priority, doctor's specialty choosing, bed allocation and billing.

In the process of development, a number of C programming concepts were applied such as arrays, strings, functions, pointers, loops, conditional statements, sorting and calculations and file handling. The step-by-step development and testing of the system allows catching errors and improving reliability of the program.

The final system fulfills main requirements of the task and proposes an easy and organized way to keep information about patients and hospital resources. The project also provided good hands-on experience in modular programming and debugging.